

Chapter 2 – section 3

Objects and Classes

Objectives

- To define methods with formal parameters.
- To invoke methods with actual parameters (i.e., arguments).
- To use method overloading and understand ambiguous overloading.
- To use UML graphical notation to describe classes and objects.
- To demonstrate how to define classes and create objects.
- To define a reference variable using a reference type and assign default values for an object's data fields.
- To distinguish between instance and static variables and methods.
- To access an object's data and methods using object reference variables and the object member access operator- dot (.).
- To define private data fields with appropriate get and set methods.
- To encapsulate data fields to make classes easy to maintain.

Methods

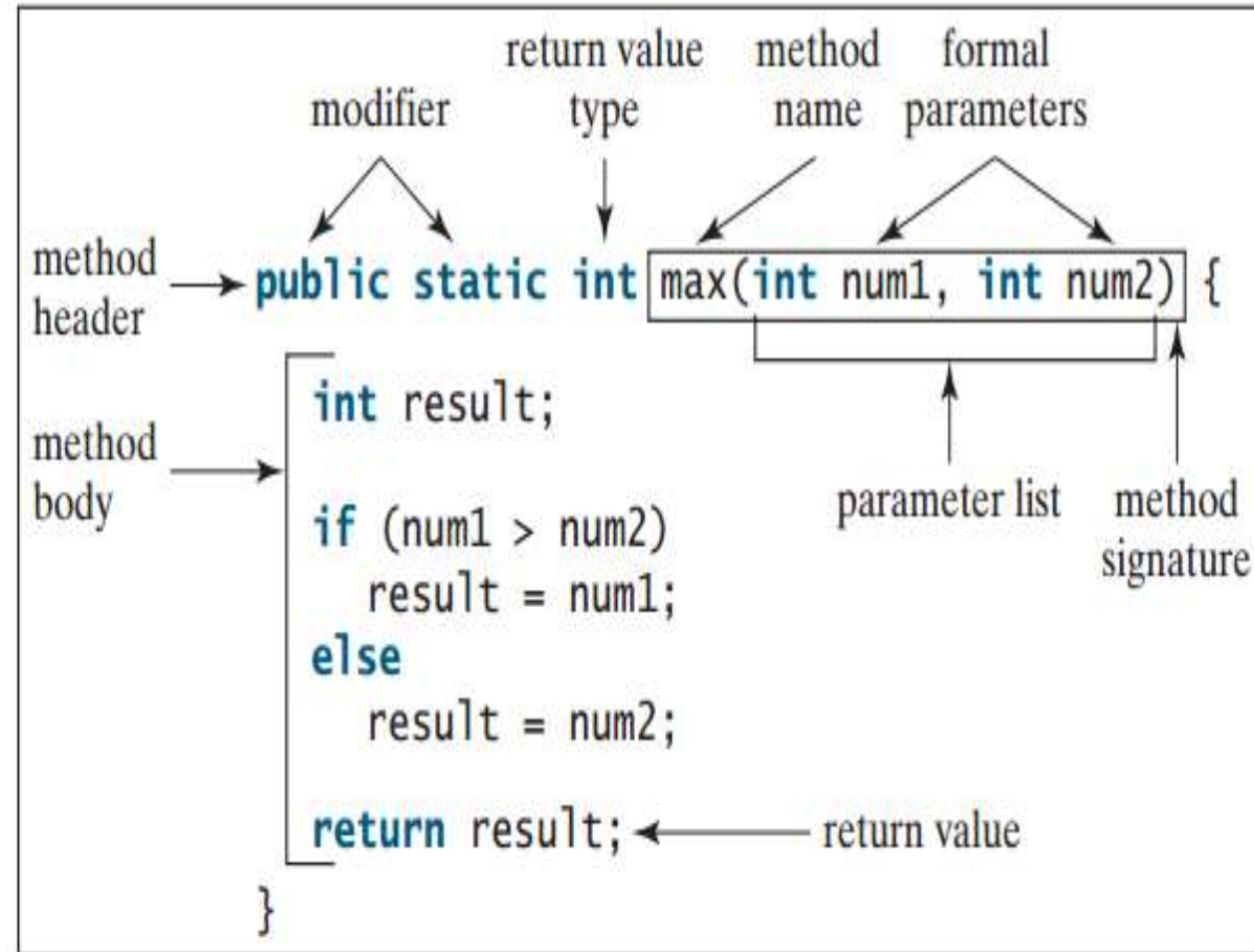
- A method is a collection of statements that are grouped together to perform an operation.
- Benefits of methods:
 - ✓ Write a method once and share or reuse it anywhere.
 - ✓ To reduce redundant code and complexity.
 - ✓ Information hiding. Hide the implementation from the user.
- Common methods that can be defined in a class:
 - **Instance method,**
 - **Static method,**
 - **Abstract method,**
 - **Final method**

Defining Methods

- A method definition consists of its modifier, method name, parameters, return value type, and body.
- In a method definition, you define what the method is to do.
- The syntax for defining a method is as follows:

```
modifier returnValueType methodName(list of parameters)  
{  
// Method body;  
}
```

Define a method



Invoke a method

```
int z = max(x, y);
```

actual parameters (arguments)

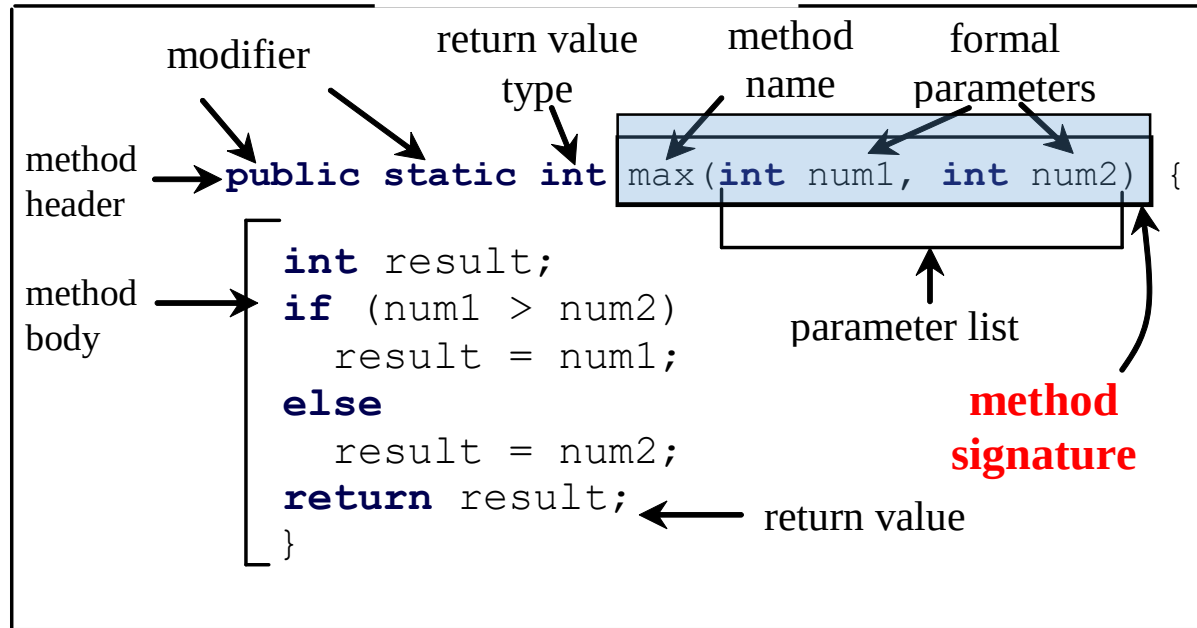
The diagram shows the invocation of the `max` method. The code `int z = max(x, y);` is shown. The `max` part is the method name, and `x, y` are the actual parameters (arguments) passed to the method. Arrows point from the text "actual parameters (arguments)" to `x` and `y`.

A method definition consists of a method header and a method body.

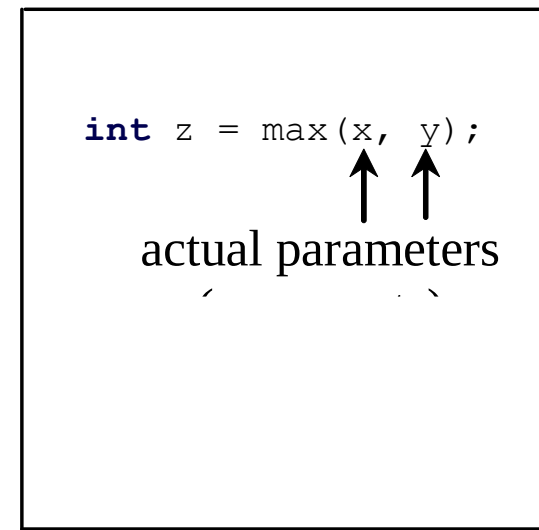
Method Signature

➤ *Method signature* is the combination of the method name and the parameter list.

Define a method

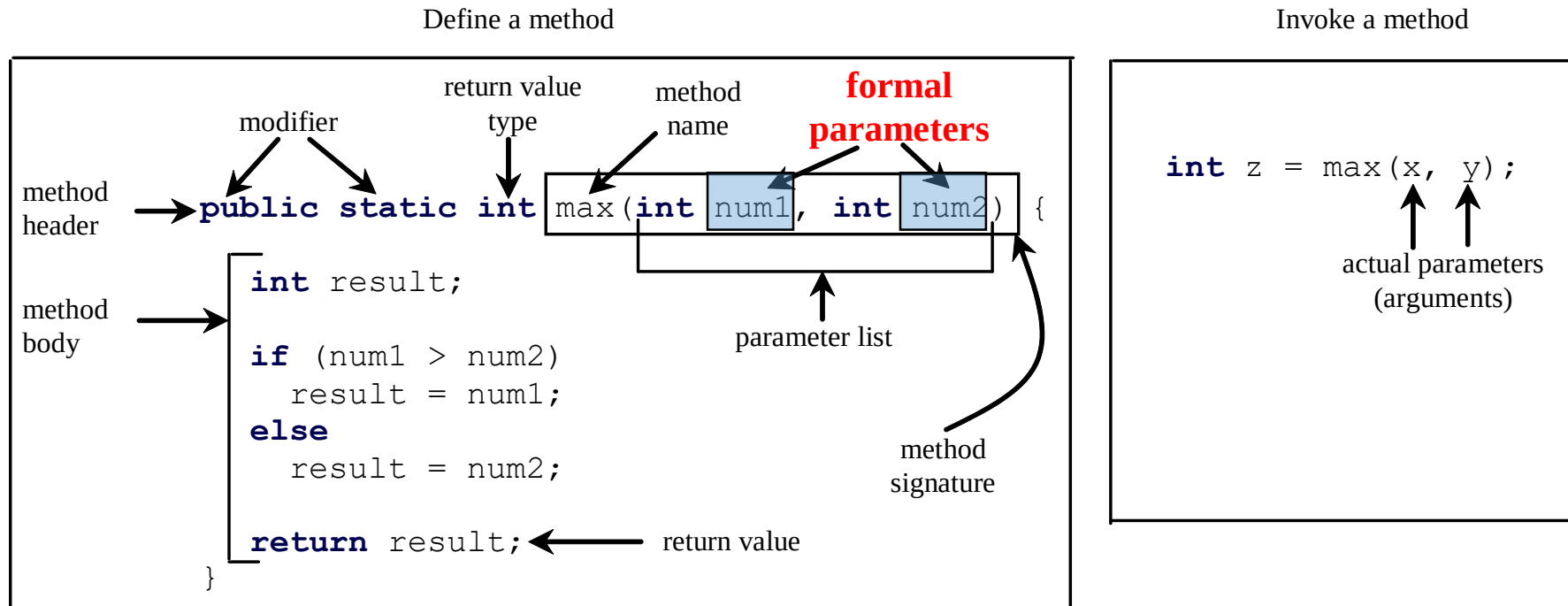


Invoke a method



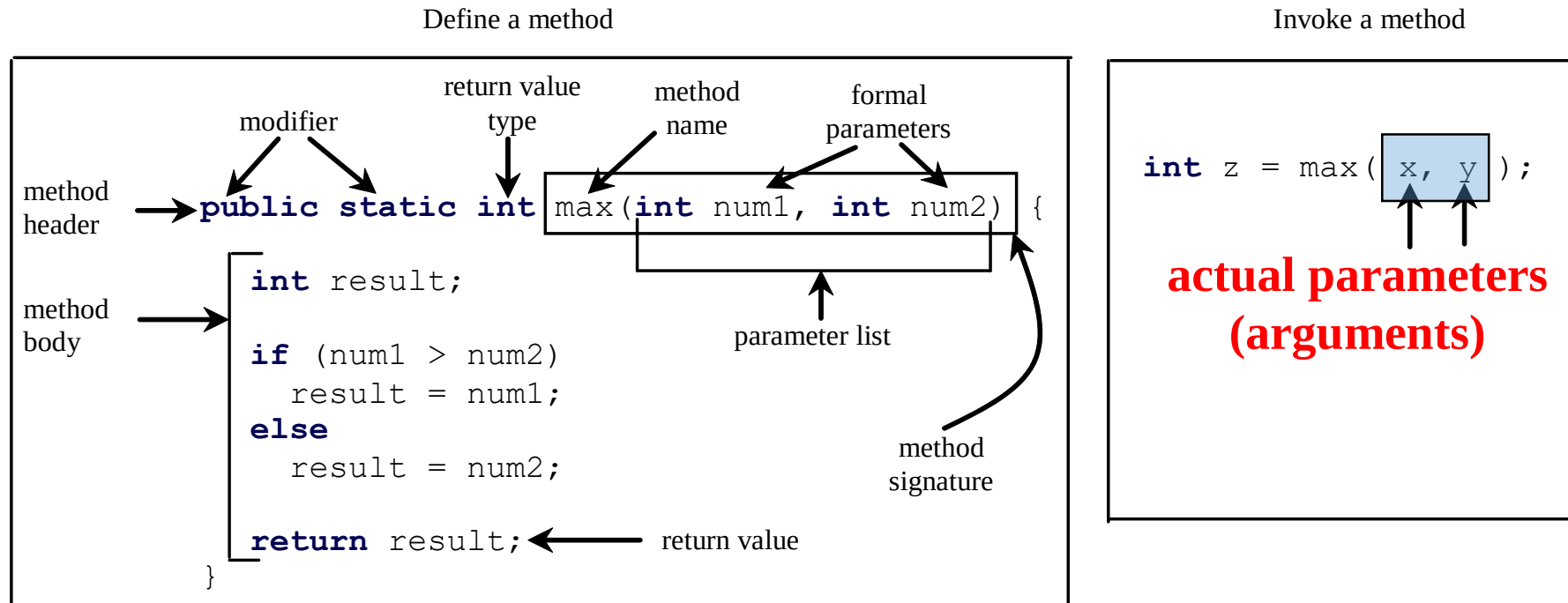
Formal Parameters

➤ The variables defined in the method header are known as *formal parameters*.



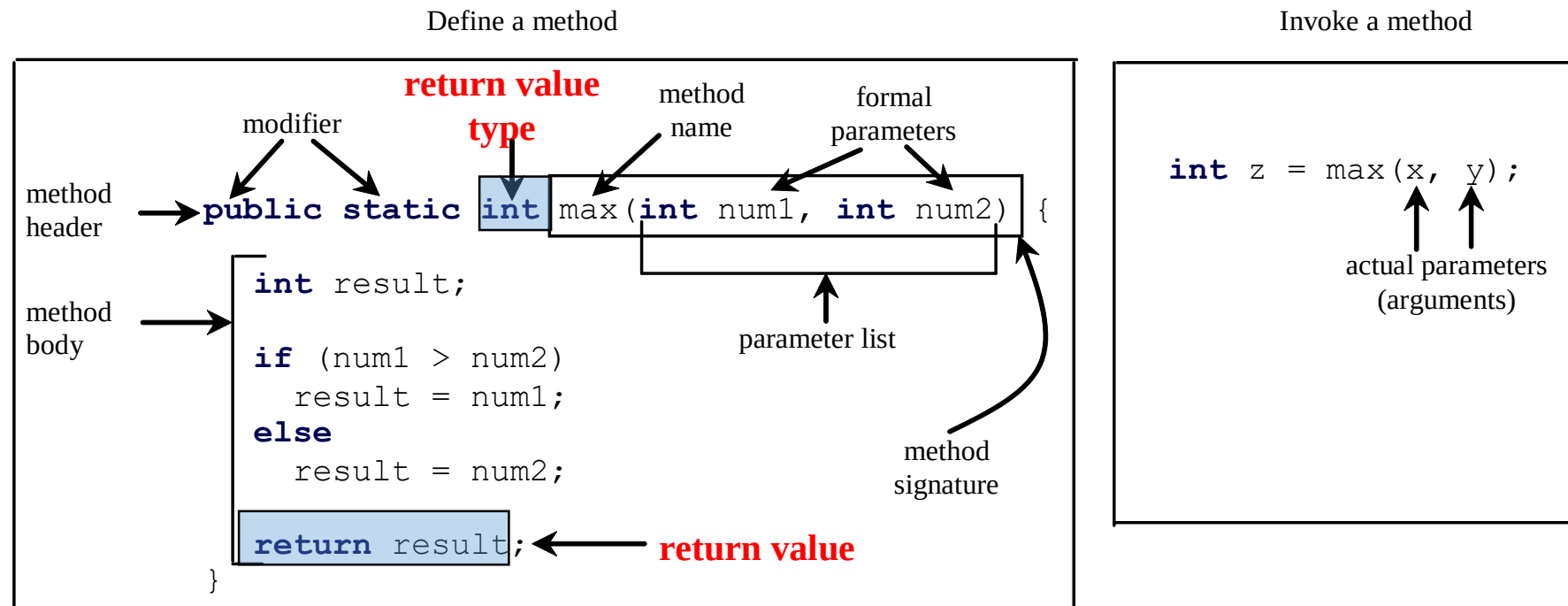
Actual Parameters

- When a method is invoked, you pass a value to the parameter.
- This value is referred to as *actual parameter* or *argument*.



Return Value Type

- The `returnValueType` is the data type of the value the method returns.
- If the method does not return a value, the `returnValueType` is the keyword `void`. For example, the `returnValueType` in the `main` method is **`void`**.



Calling a Method

- To execute the codes in a method, you have to call or invoke it.
- There are three ways to call a method:
 1. Using a method name by itself to call another method of the same class.
 2. Using a variable that contains a reference to an object, followed by a dot (.) and the method name to call a **non static method** of the referenced object.
 3. Using the class name and a dot (.) to call a **static method** of a class.
- ☞ Static methods are not tied to a specific object.

Calling a Method...

- If a method returns a value, a call to the method is usually treated as a value.

For example:

```
int larger = max(3,4);
```

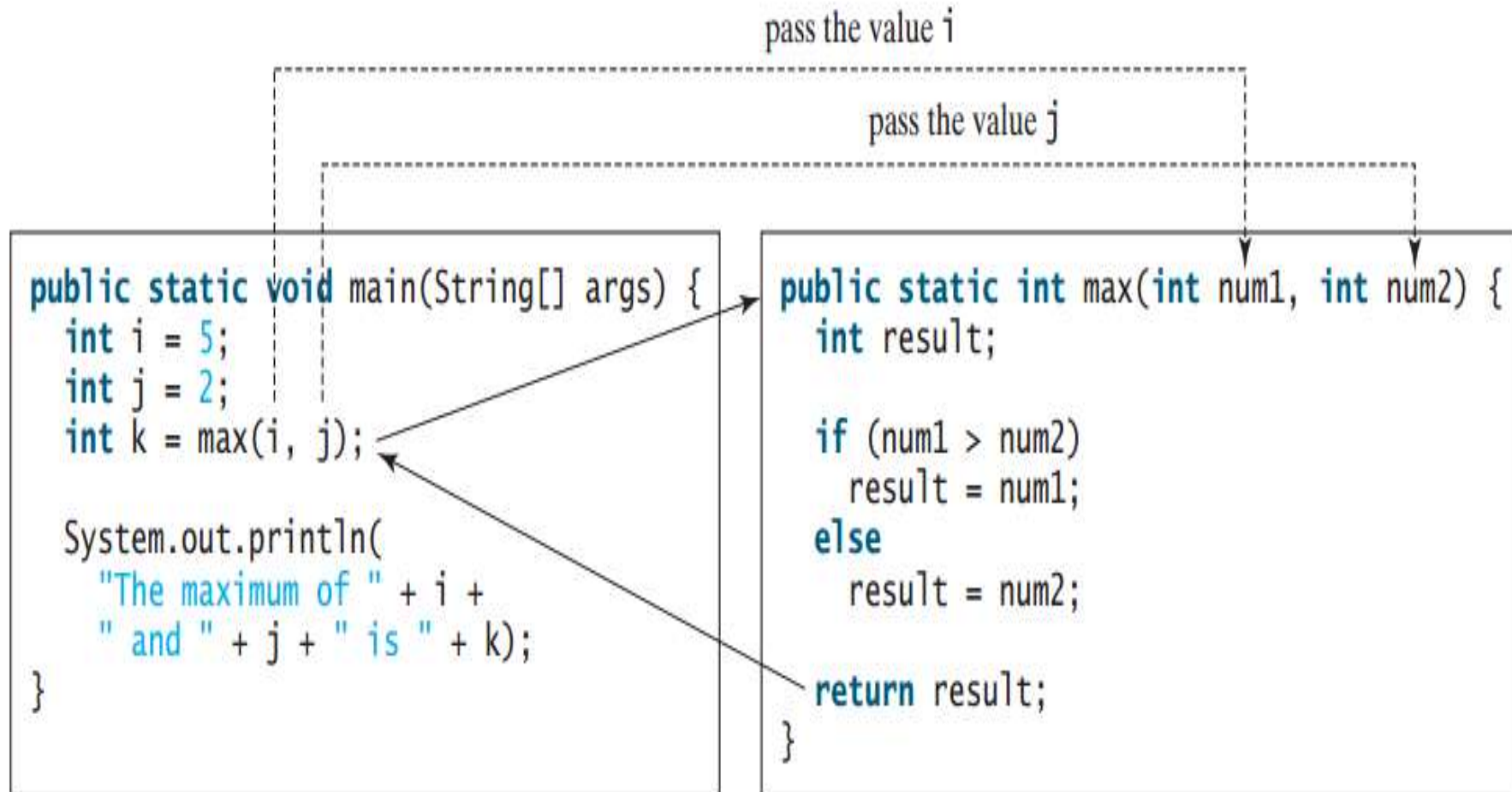
```
System.out.println(max(3,4));
```

- If a method returns void, a call to the method must be a statement.

For example,

```
System.out.println("Welcome to Java!");
```

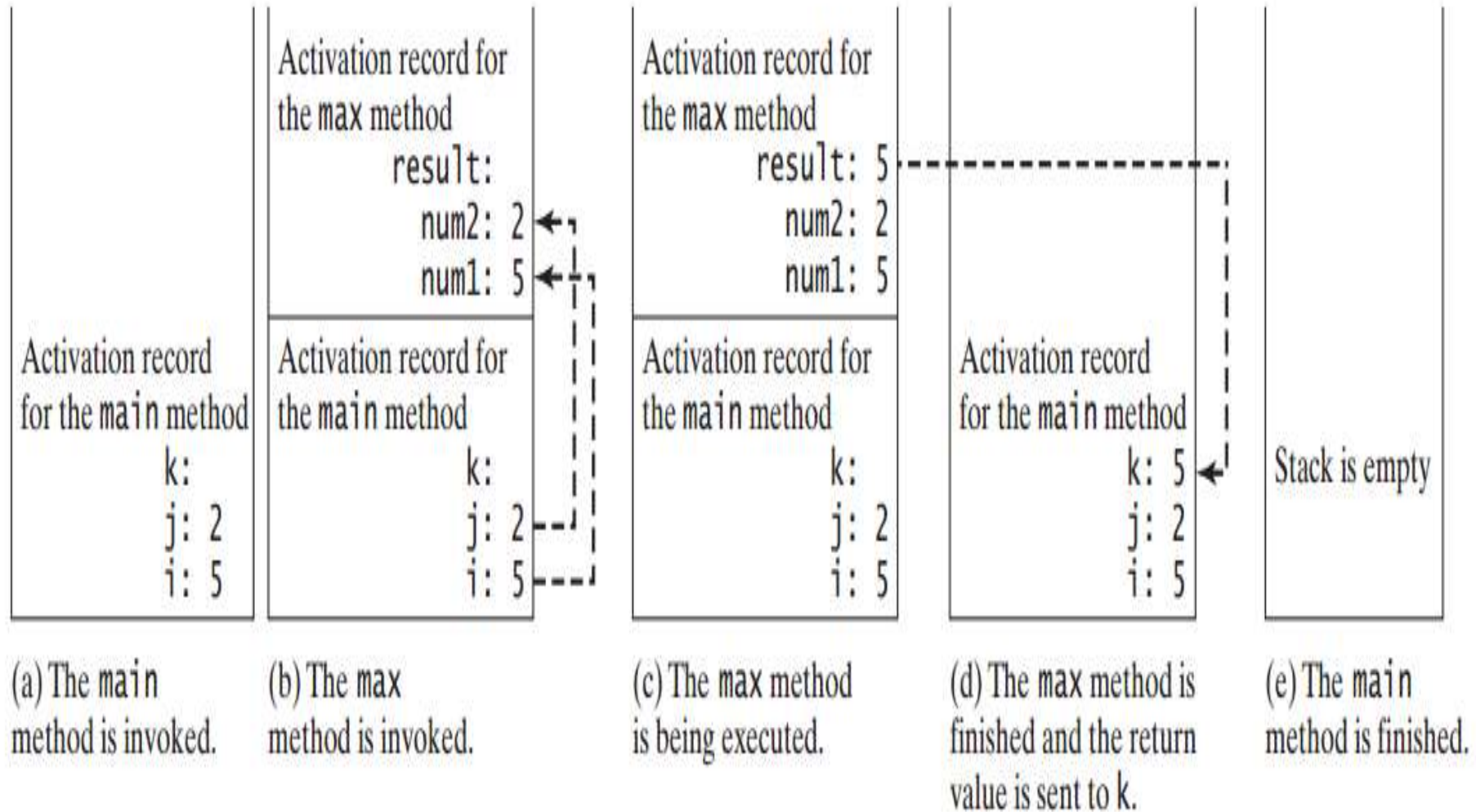
Calling a Method(cont'd...)



Method Call Stacks

- Each time a method is invoked, the system creates an **activation record**(also called an **activation frame**) that stores parameters and variables for the method and places the activation record in an area of memory known as a call stack or simply stack.
- A call stack stores the activation records in a last-in, first-out fashion: The activation record for the method that is invoked last is removed first from the stack.

Method Call Stacks...



void Method

- This type of method does not return a value.
- The void method performs some actions when invoked and return nothing.

Example:

```
public static void printGrade(double score) {  
    if(score >= 90.0)  
        System.out.println('A');  
    else if(score >= 80.0)  
        System.out.println('B');  
    ...  
}
```

Passing Parameters by Value

- When you invoke a method with an argument, the value of the argument is passed to the parameter.
 - ☞ This is referred to as **pass-by-value**.
- Java supports pass by value only.
- The arguments must match the parameters in order, number, and compatible type, as defined in the method signature.

Passing Parameters by Value...

Example1:

```
public static void nPrintln(String message, int n){  
    for (int i = 0; i < n; i++)  
        System.out.println(message);  
}
```

➤ What will be the output?

1. If you invoke the method using:
 - `nPrintln("Welcome to Java", 5);`
 2. Suppose you invoke the method using:
 - `nPrintln("Computer Science", 15);`
- Can you invoke the method using:
- `nPrintln(15, "Computer Science");` ?

Method Overloading

- Overloading methods enables you to define methods with the same name in the same class as long as their signatures are different (different sets of parameters determined by the number, types and order of the parameters).
- You cannot overload methods based on different modifiers or return types.
- Methods that perform the same function with different types of parameters should be given the same name.

Ambiguous Invocation

- Sometimes there may be two or more possible matches for an invocation of a method, but the compiler cannot determine the most specific match.
- This is referred to as *ambiguous invocation*.
- Ambiguous invocation is a compile time error.

Example:

```
1 public static void main(String[] args) {  
2     System.out.println(max(1, 2));  
3 }  
4  
5 public static double max(int num1, double num2){  
6     if (num1 > num2)  
7         return num1;  
8     else  
9         return num2;  
10 }  
11 public static double max(double num1, int num2){  
12     if (num1 > num2)  
13         return num1;  
14     else  
15         return num2;  
16 }  
17
```

Local Variables

- Local variables are declared in methods or constructors.
- Local variables are created when the method or constructor is entered and the variable will be destroyed once it exits the method or constructor.
- Access modifiers cannot be used for local variables.
- Local variables are visible only within the declared method or constructor.
- There is no default value for local variables so local variables should be declared and an initial value should be assigned before the first use.

Local Variables...

Example:

```
public class Test {  
    public static void main(String[] args) {  
        int x; // x has no default value  
        String y; // y has no default value  
        System.out.println("x is " + x);  
        System.out.println("y is " + y);  
    }  
}
```



Compile error: variable not initialized

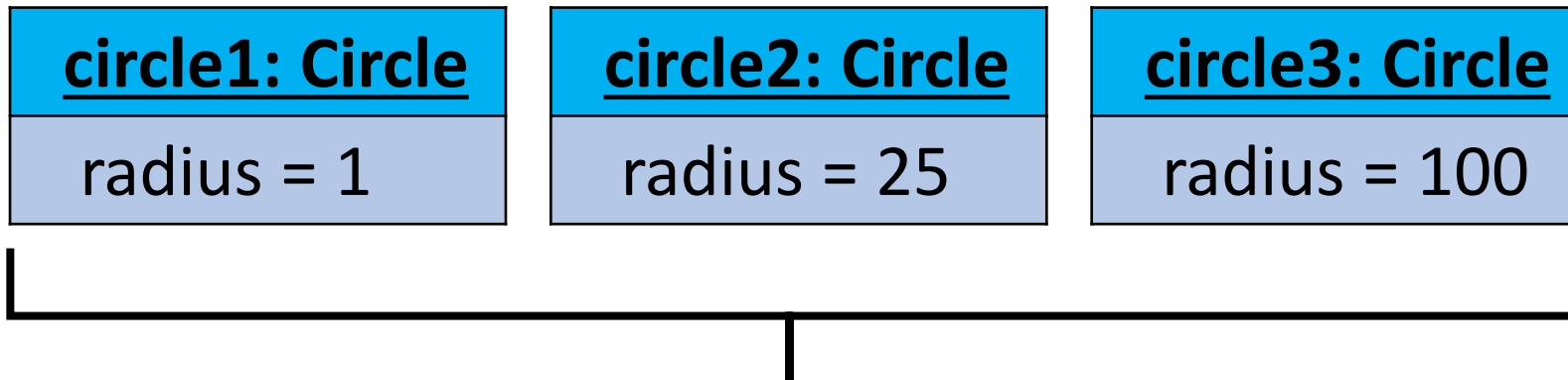
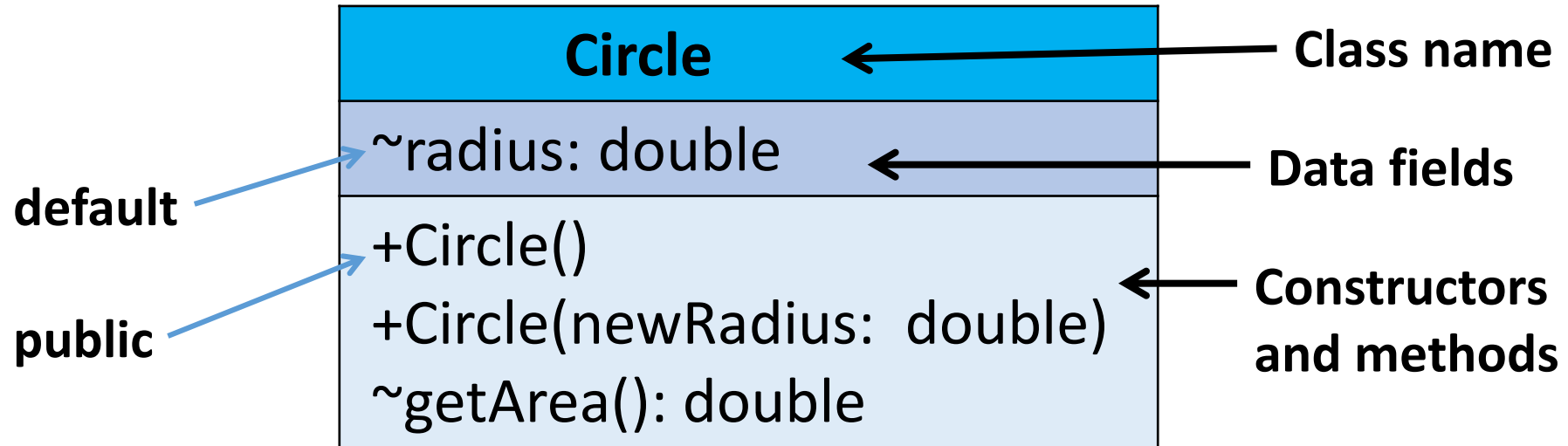
Another Example

Objects and Classes

More on Objects and Classes

- An *object* represents an entity in the real world that can be distinctly identified.
- An object has both a state and behavior.
- The state defines the object (properties), and the behavior defines what the object does.
- *Classes* are constructs that define objects of the same type.
- A Java class uses variables to define data fields and methods to define behaviors.
- We can use UML class diagram to show the classes, fields, methods and relationship among objects.

UML Class Diagram



UML notation for objects

UML Class Diagram to Java Code

```
public class Circle {  
    double radius = 1.0;   
    // Construct a circle object  
    public Circle() {  
    }  
    // Construct a circle object  
    public Circle(double newRadius) {  
        radius = newRadius;  
    }  
    // Return the area of this circle  
    double getArea() {  
        return radius * radius * Math.PI;  
    }  
}
```

Data field

Constructors

Method

Defining Classes and Creating Objects

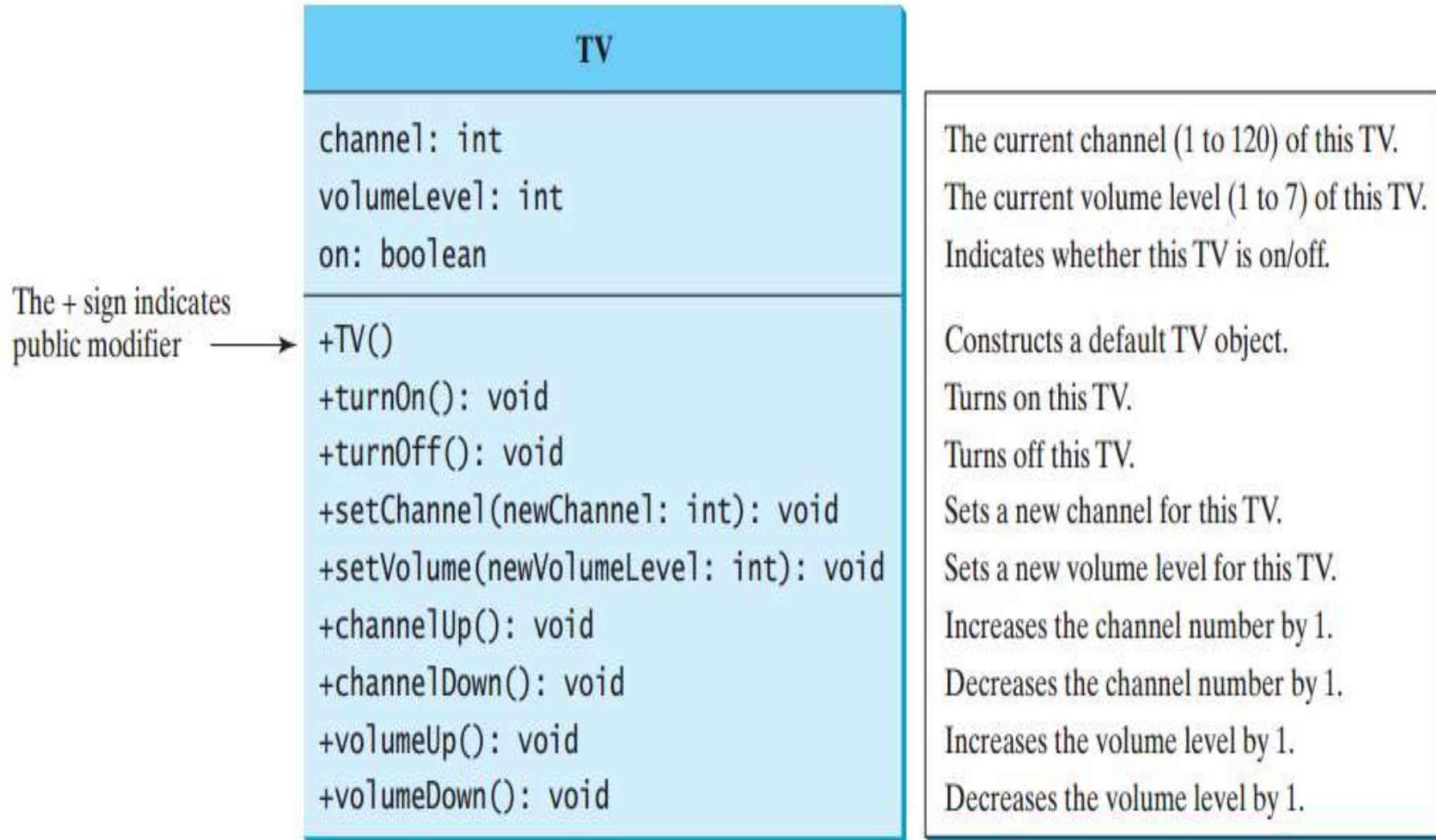
- Classes are definitions for objects and objects are created from classes.
- Objects can be created by constructors.

Problem statement:

consider television sets. Each TV is an object with states (current channel, current volume level, power on or off) and behaviors (change channels, adjust volume, turn on/off). You can use a class to model TV sets. The UML diagram for the class is shown in figure(next slide).

- Objective: Demonstrate creating objects, accessing data, and using methods.

Defining Classes and Creating Objects...



Run

Differences between object and class

Object:

- Object is an instance of a class.
- Object is a real world entity.
- Object is a physical or logical entity.

Class:

- Class is a blueprint from which objects are created.
- Class is a group of similar objects.
- Class is a logical entity.

Differences between object and class...

Object:

- Object is created many times as per requirement.
- Object allocates memory when it is created.
- There are many ways to create object in java such as new keyword, newInstance() method, clone() method, factory method.

Class:

- Class is declared once.
- Class doesn't allocated memory when it is created.
- There is only one way to define class in java using class keyword.

Constructors

- Constructors are special kind of methods that are invoked to construct and initialize objects.
- Constructors have three peculiarities:
 - ✓ must have the same name as the class itself.
 - ✓ do not have a return type—not even void.
 - ✓ are invoked using the **new** operator when an object is created.

Creating Objects Using Constructors

- To construct an object from a class, invoke a constructor of the class using the new operator, as follows:

new ClassName();

or

new ClassName(arguments);

Example:

new Circle();

new Circle(5.0);

Default Constructor

- A class may be defined without constructors.
- In this case, a no-arg constructor with an empty body is implicitly defined in the class by the compiler.
- This constructor, called *a default constructor*, is provided automatically *only if no constructors are explicitly defined in the class*.

Overloaded Constructors

- Overloaded constructors enable objects of a class to be initialized in different ways.
- The compiler differentiates overloaded constructors by their signatures.
- To call one constructor of a class from another of the same class, you can use the **this** keyword followed by parentheses containing the constructor arguments.
- If used, such a constructor call must appear as the first statement in the constructor's body.

The this Keyword

- The **this** keyword is the name of a reference that refers to an object itself.
- One common use of the this keyword is reference a class's *hidden data fields*.
- Another common use of the this keyword is to enable a constructor to invoke another constructor of the same class.
 - 👉 To invoke overloaded constructors.

Reference the Hidden Data Fields

```
public class F {  
    private int i = 5;  
    private static double k = 0;  
    public void setI(int i) {  
        this.i = i;  
    }  
    public static void setK(double k) {  
        F.k = k;  
    }  
    // Other methods omitted  
}
```

- Suppose that f1 and f2 are two objects of F.
- ✓ Invoking f1.setI(10) is to execute:
 this.i = 10, where this refers f1
- ✓ Invoking f2.setI(45) is to execute:
 this.i = 45, where this refers f2
- ✓ Invoking F.setK(33) is to execute:
 F.k = 33, setK is a static method

Calling Overloaded Constructor

```
public class Circle {  
    private double radius;  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
    public Circle( ) {  
        this(1.0);  
    }  
    public double getArea( ) {  
        return this.radius * this.radius * Math.PI;  
    }  
}
```

this must be explicitly used to reference the data field radius of the object being constructed

this is used to invoke another constructor

Every instance variable belongs to an instance represented by **this**, which is normally omitted

Declaring Object Reference Variables

➤ To reference an object, assign the object to a reference variable.

➤ To declare a reference variable, use the syntax:

ClassName objectRefVar;

Example: **Circle myCircle;**

➤ To declare and create objects in a single step:

ClassName objectRefVar = new ClassName();

Example: Assign object reference Create an object

Circle myCircle = new Circle();



Instance Variables

- Instance variables are declared in a class, but outside a method, constructor or any block.
- Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed.
- Access modifiers can be given for instance variables.
- Instance variables belong to a specific instance and have memory storage independent of one another.
- Each object (instance) of the class has its own copy of the class's instance variables.

Instance Variables...

- Instance variables have default values.
- Values can be assigned during the declaration or within the constructor.
- Instance variables can be accessed directly by calling the variable name inside the class.
- However within static methods and different class instance variables should be accessed using the fully qualified name . *ObjectReference.VariableName*.

Static Variables and Constants

- Static variables also known as class variables are declared with the ***static*** keyword in a class, but outside a method, constructor or a block.
- Static variables are shared by all the instances of the same class.
- There would only be one copy of each class variable per class, regardless of how many objects are created from it.
- Visibility and default values are same as instance variables.

Static Variables and Constants...

- Values can be assigned during the declaration or within the constructor.
- Most static variables are declared public since they must be available for users of the class.
- Static variables can be accessed by calling with the class name.

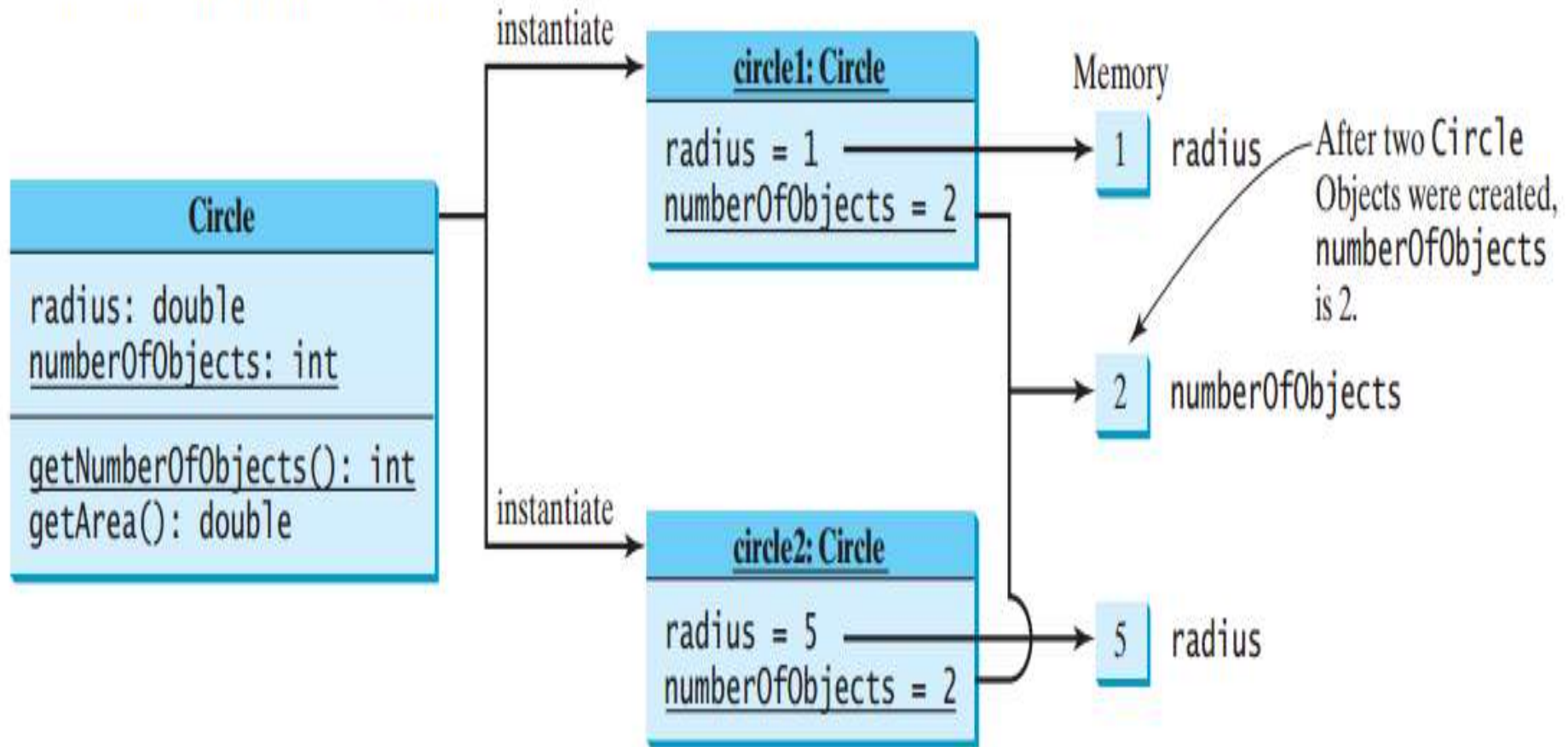
ClassName.VariableName.

- Static constants are final variables shared by all the instances of the class.
- To declare any static members of a class, use the **static** modifier.

Static Variables, Constants and Methods...

UML Notation:

underline: static variables or methods



Scope of Variables

- ❑ The scope of instance and static variables is the entire class.
- ❑ The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable.

Relationship between static and instance members

Instance method can:

- invoke an instance method.
- access an instance data field.
- invoke a static method.
- Access a static data field.

Static method can:

- **not** invoke an instance method.
- **not** access an instance data field.
- invoke a static method.
- Access a static data field.

Java Modifiers

- Like other languages, it is possible to modify classes, methods, etc., by using modifiers.
- There are two basic categories of modifiers:
 - 1. Access Modifiers:** default, public , protected, private
 - 2. Non-access Modifiers:** final, abstract, static, ...

Access / Visibility Modifiers

➤ The common access modifiers are:

- **public:** The class, data, or method is visible to any class in any package.
- **protected:** A protected member of a class can be accessed from a subclass.
- **private:** The data or methods can be accessed only by the declaring class.
 - ✓ The get and set methods are used to read and modify private properties.
- **default:** the class, variable, or method can be accessed by any class in the same package.

Data and Methods Visibility

Modifiers on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass in a different package	Accessed from a different package
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
Default (no modifier)	Yes	Yes	No	No
private	Yes	No	No	No

Example:

```
package p1;

public class C1 {
    public int x;
    int y;
    private int z;

    public void m1() {
    }
    void m2() {
    }
    private void m3() {
    }
}
```

```
package p1;

public class C2 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        can access o.y;
        cannot access o.z;

        can invoke o.m1();
        can invoke o.m2();
        cannot invoke o.m3();
    }
}
```

```
package p2;

public class C3 {
    void aMethod() {
        C1 o = new C1();
        can access o.x;
        cannot access o.y;
        cannot access o.z;

        can invoke o.m1();
        cannot invoke o.m2();
        cannot invoke o.m3();
    }
}
```

```
package p1;

class C1 {
    ...
}
```

```
package p1;

public class C2 {
    can access C1
}
```

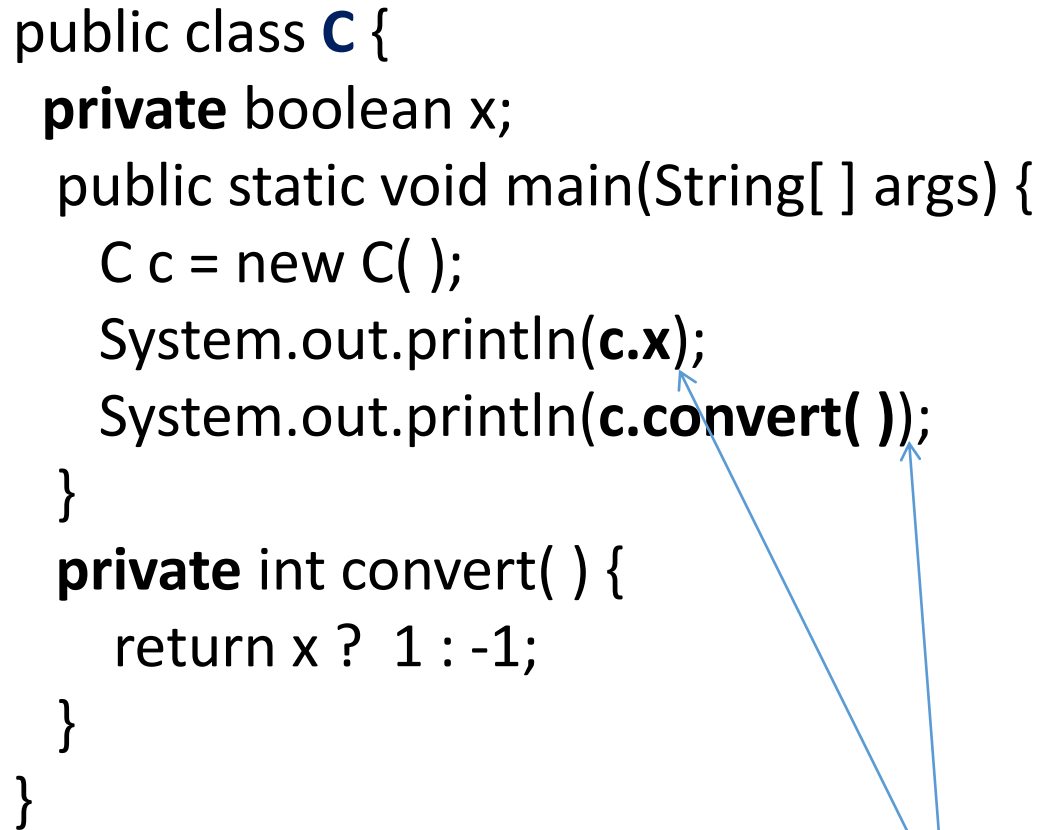
```
package p2;

public class C3 {
    cannot access C1;
    can access C2;
}
```


Cont'd...

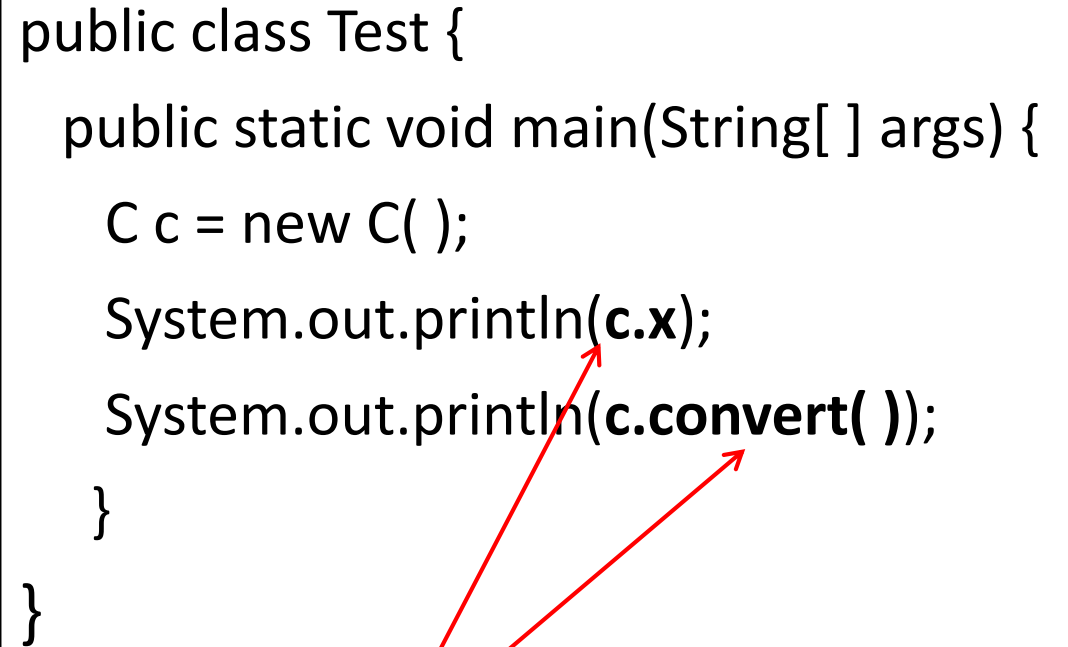
- An object cannot access its private members, as shown in (b).
- But it is OK if the object is declared in its own class, as in (a).

```
public class C {  
    private boolean x;  
    public static void main(String[ ] args) {  
        C c = new C( );  
        System.out.println(c.x);  
        System.out.println(c.convert( ));  
    }  
    private int convert( ) {  
        return x ? 1 : -1;  
    }  
}
```



(a) This is ok because object **c** is used inside the class **C**.

```
public class Test {  
    public static void main(String[ ] args) {  
        C c = new C( );  
        System.out.println(c.x);  
        System.out.println(c.convert( ));  
    }  
}
```



(b) This is wrong because **x** and **convert** are private in class **C**.

Accessing Object's Members

- Referencing the object's instance data:

objectRefVar.data

e.g., myCircle.radius;

- Invoking the object's instance method:

objectRefVar.methodName(arguments)

e.g., myCircle.getArea();

Note: use the class name to reference static data or to invoke static method.

Example

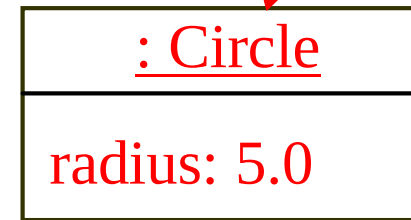
```
Circle myCircle = new Circle(5.0);
```

```
Circle yourCircle = new Circle();
```

```
yourCircle.radius = 100;
```

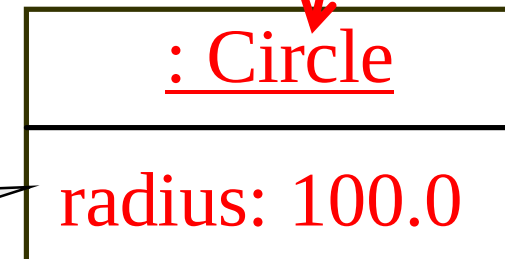
myCircle

reference value



yourCircle

reference value



Change radius in yourCircle

Reference Data Fields

- The data fields can be of reference types.
- If a data field of a reference type does not reference any object, the data field holds a special literal value, null.

For example, the following Student class contains a data field **name** of the String type.

Example:

```
public class Student {  
    String name; // name has default value null  
    int age; // age has default value 0  
    boolean isScienceMajor; // has default value false  
    char gender; // has default value '\u0000'  
}
```

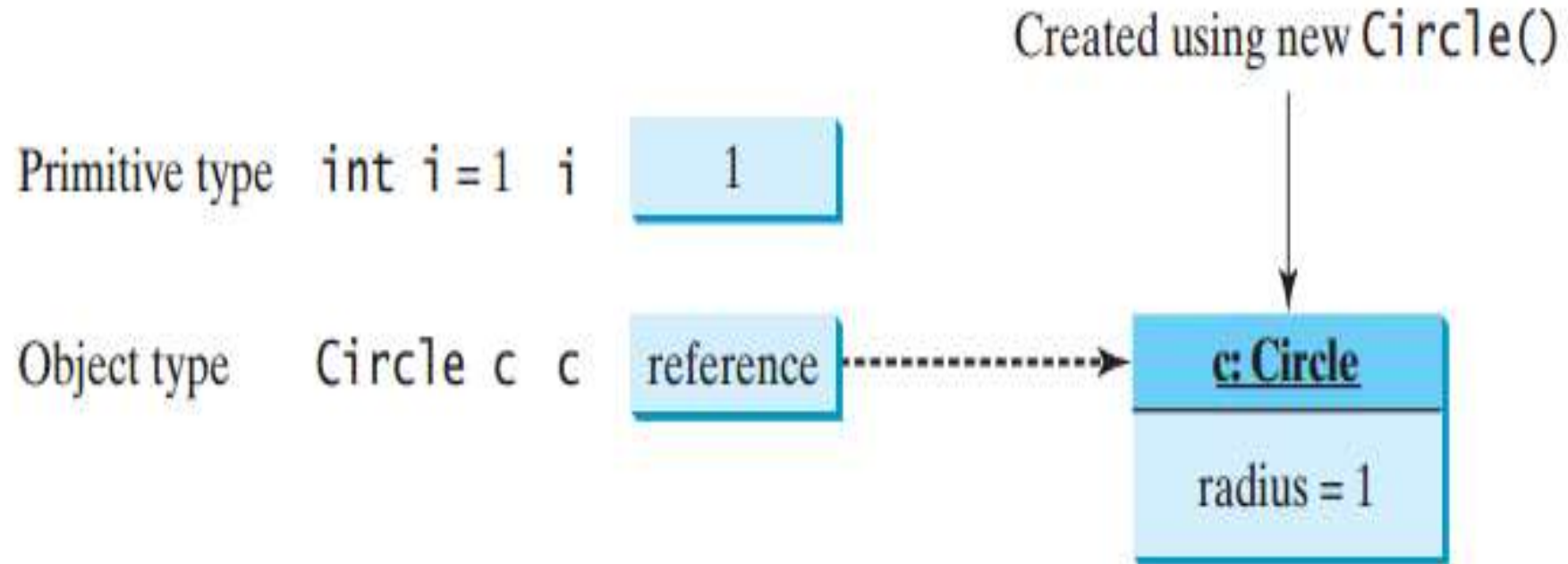
Example Cont'd...

```
public class Test {  
    public static void main(String[] args) {  
        Student student1 = new Student();  
        System.out.println("name: " + student1.name);  
        System.out.println("age: " + student1.age);  
        System.out.println("isScienceMajor? " +  
                             student1.isScienceMajor);  
        System.out.println("gender: " + student1.gender);  
    }  
}
```

What will be the output of the above code?

```
name: null  
age: 0  
isScienceMajor? false  
gender:
```

Differences between Variables of Primitive Data Types and Object Types



Copying Variables of Primitive Data Types

```
int i = 1;  
int j = 2;
```

Primitive type assignment $i = j$

Before:

i 1

j 2

After:

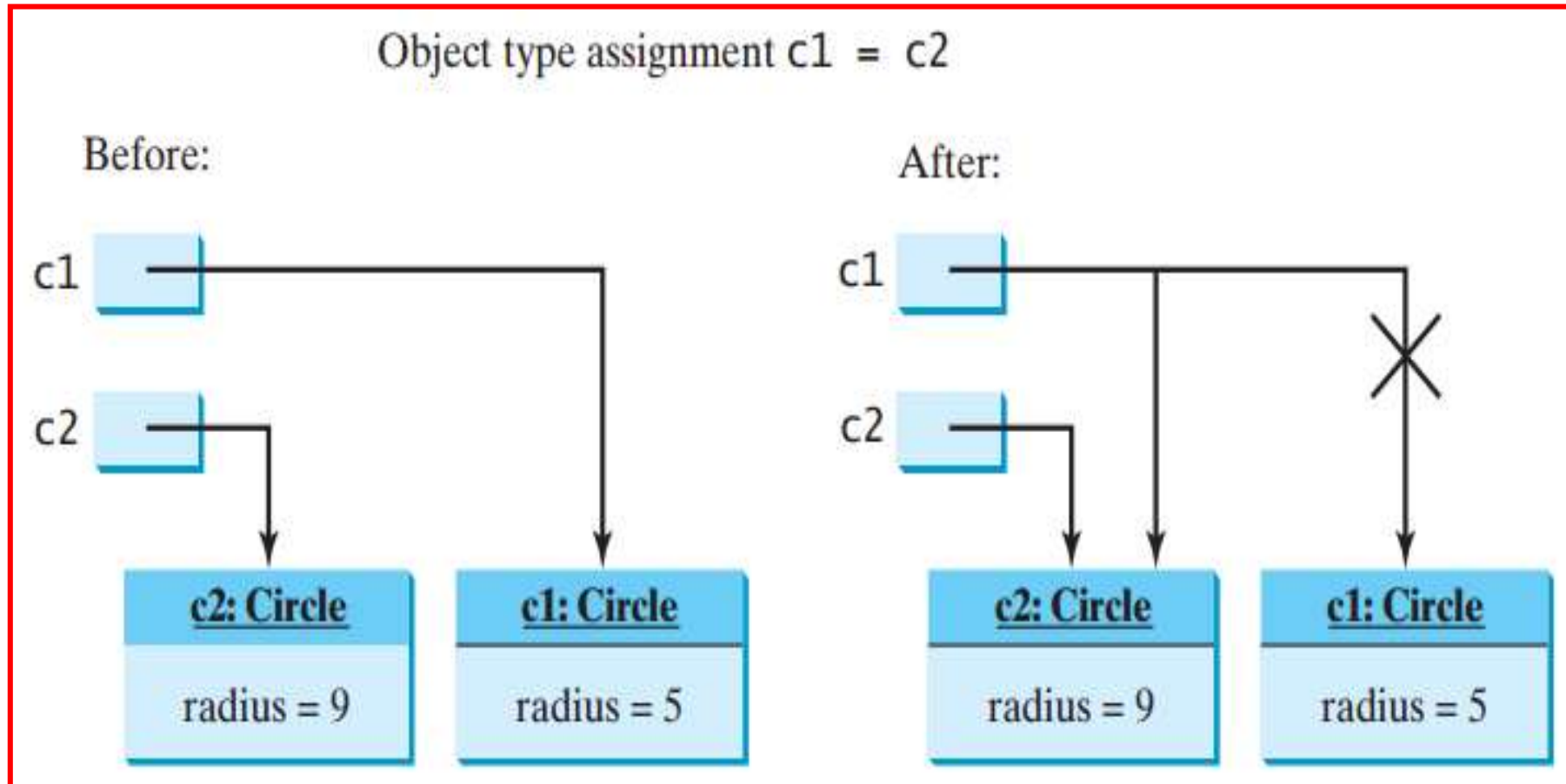
i 2

j 2

Copying Variables of Object Types

Circle c1 = new Circle(5.0); // with radius = 5

Circle c2 = new Circle(9.0); // with radius = 9



Data Field Encapsulation

➤ To prevent direct modifications of data fields, you should declare the data fields private, using the private modifier.

☞ This is known as data field encapsulation.

➤ Why data fields should be private?

✓ To protect data.

✓ To make code easy to debug and maintain.

➤ Classes can encapsulate their properties and provide getter/setter and other methods.

Passing Objects to Methods

- Passing an object to a method is to pass the reference of the object.
- Java uses exactly one mode of passing arguments: pass-by-value.
- Passing by value for reference type value (the value is the reference to the object).

Example

```
1 class StudentInfo {  
2     double age;  
3     String name;  
4     public StudentInfo(String name, double age){  
5         this.name = name;  
6         this.age = age;  
7     }  
8  
9     void displayStudentInfo(StudentInfo s) {  
10        System.out.println("Student Name: " + s.name);  
11        System.out.println("Student Age: " + s.age);  
12    }  
13 }  
14  
15 public class PassObjects {  
16     public static void main(String[] args) {  
17         StudentInfo stu = new StudentInfo("Abebe", 30);  
18         stu.displayStudentInfo(stu); // Pass object to method  
19     }  
20 }
```

• Output????

Student Name: Abebe

Student Age: 30.0

Garbage Collector

- In java, garbage means unreferenced objects.
- The garbage collector is a program which runs on the Java Virtual Machine which gets rid of the unreferenced objects .
- It is a form of *automatic memory management*.
- Advantage of Garbage Collection:
 - ✓ It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.
 - ✓ It is **automatically done** by the garbage collector(a part of JVM) so we don't need to make extra efforts.

Garbage Collector...

- The **finalize()** method in Object class is invoked each time before the object is garbage collected to perform cleanup processing.
- The **gc()** method in System and Runtime classes is used to invoke the garbage collector to perform cleanup processing.
- Note: Garbage collection is performed by a daemon thread called Garbage Collector(GC). This thread calls the finalize() method before object is garbage collected.

How can an object be unreferenced?

1) By nulling a reference:

```
Employee e=new Employee();  
e=null;
```

2) By assigning a reference to another:

```
Employee e1=new Employee();  
Employee e2=new Employee();
```

//now the 1st object referred by e1 is available for garbage collection

```
e1=e2;
```

3) By anonymous object:

```
new Employee();
```

Immutable Objects and Classes

- If the contents of an object cannot be changed once the object is created, the object is called an *immutable object* and its class is called an *immutable class*.
- A class with private fields and no mutators can be immutable.
- However a class with all private data fields and without mutators is not necessarily immutable.
- For a class to be immutable, it must mark all data fields private and provide no mutator methods and no accessor methods that would return a reference to a mutable data field object.

Example

```
public class MutablePoint {  
    int x; // not private, not final  
    MutablePoint(int x) {  
        this.x = x;  
    }  
  
    public int getX() {  
        return x;  
    }  
  
    public static void main(String[] args) {  
        MutablePoint p = new MutablePoint(10);  
        System.out.println("Original value: " + p.getX());  
  
        p.x = 99;  
        System.out.println("After change: " + p.getX());  
    }  
}
```

- Output??

Original value: 10

After change: 99

```
public class ImmutablePoint {  
    private final int x; // private + final  
    ImmutablePoint(int x) {  
        this.x = x;  
    }  
  
    public int getX() {  
        return x;  
    }  
  
    public static void main(String[] args) {  
        ImmutablePoint p = new ImmutablePoint(10);  
        System.out.println("Original value: " + p.getX());  
  
        p.x = 99;  
        System.out.println("After trying to change: " + p.getX());  
    }  
}
```

- Output??

Error because p.x=99 is not allowed

Example

```
public class MutableExample {  
  
    private int[] scores; // mutable object (array)  
  
    MutableExample(int[] scores) {  
        this.scores = scores; // stores the original array reference  
    }  
  
    public int[] getScores() {  
        return scores; // allows external modification  
    }  
  
    public static void main(String[] args) {  
  
        int[] arr = {10, 20};  
        MutableExample obj = new MutableExample(arr);  
  
        System.out.println("Before change: " + obj.getScores()[0]);  
  
        int[] ref = obj.getScores(); // return a reference to a changable data field object (i.e scores array reference variable)  
        ref[0] = 99;  
  
        System.out.println("After change: " + obj.getScores()[0]);  
  
        /* Eventhough , a class mark all data fields private and provide no mutator methods,  
        accessor methods (get methods) that would return a reference to a changable data field object makes mutable.  
  
        */  
    }  
}
```

- Output??

Before change: 10

After change: 99